

Faculty of Engineering & Technology
Electrical & Computer Engineering Department
ENCS3310 Advanced Digital Design
First Semester – 2024/2025
Course Project

Prepared by:

Name:Abdulrahman Atyani Id:1221808

Instructor: Dr. Abdullatef Abu Issa
Date:8/12/2024
Section:4

Table of contants

Contents

Table of contants 2
Table of figuers 3
Introduction 4
Main comparator 4
Register 6
Complete code 7
Test bench 7
Simulation part and testing 8
Future work 9
Conclusion 10

Table of figuers

Figure 1:block diagram of comparator. 5
Figure 2:register storing 6-bit. 6
Figure 3:block diagram of my system. 7
Figure 4:simulation of unsigend numbers. 8
Figure 5:simulation of signed numbers. 8
Figure 6:change value to make error. 8
Figure 7:the error simulation when making error. 9

Introduction

I would like to start by talking about some of the details in my project, which aims to create a comparator between two 6-bit shelves with additions aimed at improving the code, such as adding time, registers, and also selection. The beginning is with the main comparator code.

Main comparator

In this code,we built the 6-bit comparator structurally using gates,It performs bitwise comparisons starting from the most significant bit (MSB) and combines the results using logic gates to produce the final outputs. 6-bit comparators are widely used in digital systems for sorting, decision-making, and other operations.

Equality (A = B):

- To check equality, all bits of A must be equal to the corresponding bits of B.
- The logic for each bit is:

  Eq(i) = NOT(Ai XOR Bi) for each bit i.

- If all these results are true for each bit, then A = B.

Greater than (A > B):

- Start comparing the bits from the most significant bit (MSB).
- If a higher bit of A is '1' and the corresponding bit of B is '0', then A > B is true.
- If the bits are the same, continue to the next less significant bit.

Less than (A < B):

- Similar to the greater-than case, but inverted.
- If a higher bit of A is '0' and the corresponding bit of B is '1', then A < B is true.

On the other hand,we create an input named S(selection),that it limit numbers are signed or not(if S==1,numbers are signed.Otherwise,numbers are unsigned).And for signed numbers,i make one operation for its that says they convert their to 2's complement,then we make the same comparison that we make to unsigned numbers and this photo show my talking.

```
module final_comparter(A, B, S, LT, GT, EQ);
    input [5:0] A, B;
    input S;
    output LT, GT, EQ;
    wire [5:0] A_signed, B_signed;

    assign A_signed = (S == 1) ? (~A + 1) : A;
    assign B_signed = (S == 1) ? (~B + 1) : B;

    Unsigned comp(A_signed, B_signed, LT, GT, EQ);
endmodule
```

And we make a behaveroil code to compare all result values from structural code,where are true,then code is correct.Otherwise code is false and we make sure that wire or gate are not completed.



Figure 1:block diagram of comparator.

Register

This is one of the important parts in my code,because the balance of results depends on its.This part contains 6 of D flip flips,each one store bit,that when we make it to save the values of two numbers and it contains asynchronous Reset (works at negative edge).

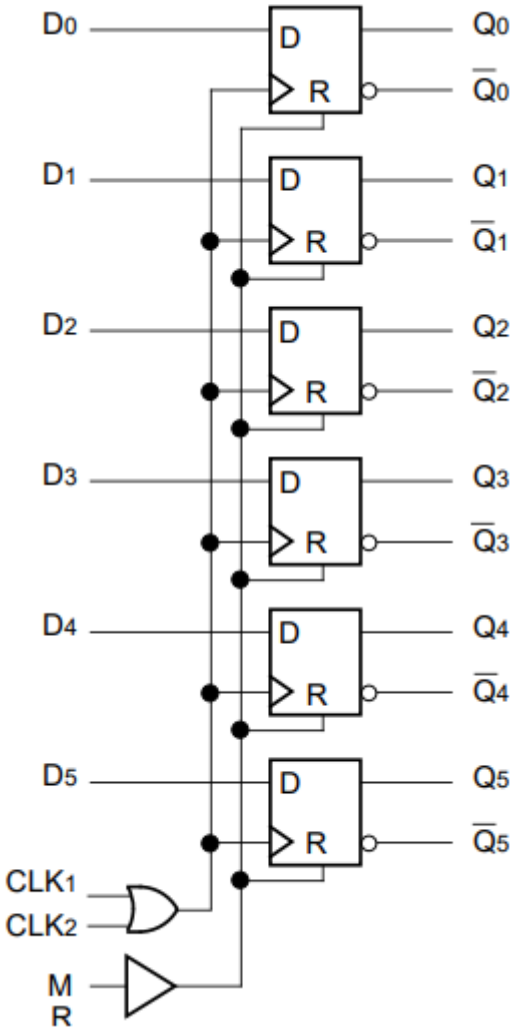


Figure 2:register storing 6-bit.

Complete code

In this part,we use the previous parts to make a full comparator that when we make one clock then the inputs move to comparator to test it,after that we make the next clock that the output will occur using 3 D filp flop to 3 outputs that each one take one of them and at the same clock we enter new inputs to comparator until all test cases finish their and this block diagram show all speaking that i talk about.

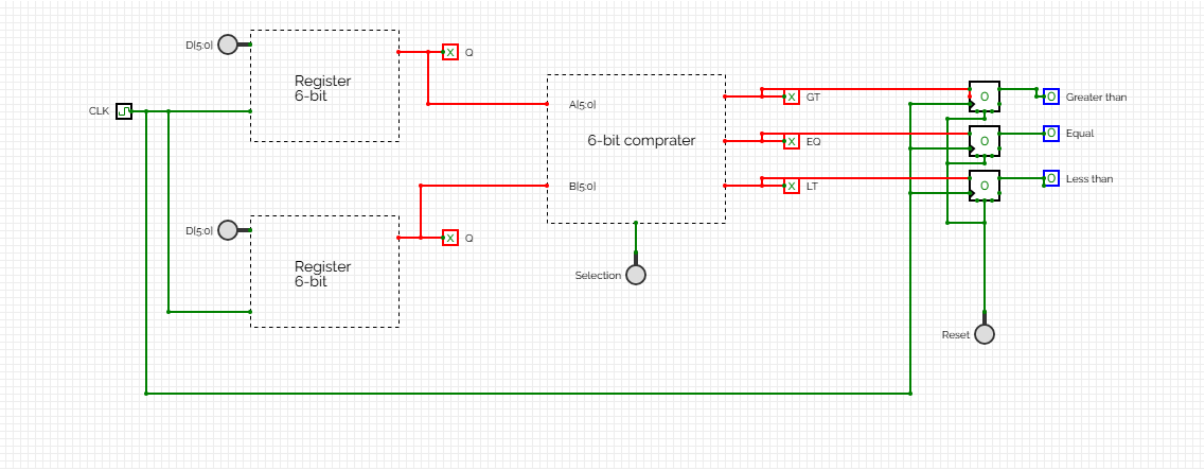


Figure 3:block diagram of my system.

Test bench

In test bench,we make all main definitions of input and outputs and we make instances of the behavioral and structural code,then we make all test cases and to make sure of all tests,i define a reg that i named T,which if one case is false then T=1.When we end of this we make if statement that says if T==1,then print fall,but if not ,it print pass.And i change the clock according to the delay in my code,where the maximum frequency is 31,this results come from delay of gates,which i add their from the table of delays in project description.

Simulation part and testing

In this section,we make testing for all possible values and all my result are excellent that the maximum latency of my work equal 31ns,this is a good number and for that the frequency value calculate it from this law (f=1/T),and the answer is 3.23MHz.

I show two photos about simulation to signed and unsigned numbers.

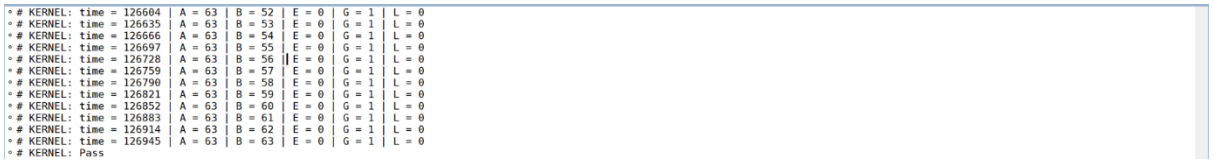


Figure 4:simulation of unsigend numbers.

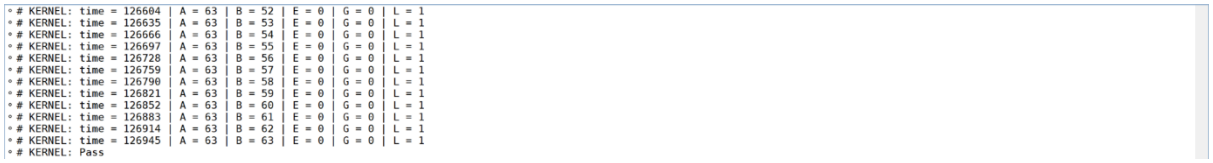


Figure 5:simulation of signed numbers.

Because of photo doesn't show all results,then i show that all result are pass in two situations,which are signed and unsigned numbers.As you see,we print the numbers,time and the output results, which is output will occur after clock of registers and the delay of code.

But if we change the location of gate then results will be different .

```
xnor #(10)(E[5], A[5], B[5]);
xnor #(10)(E[4], A[4], B[4]);
xnor #(10)(E[3], A[3], B[3]);
xnor #(10)(E[2], A[2], B[2]);
xnor #(10)(E[1], A[1], B[1]);
xor #(10)(E[0], A[0], B[0]);
```

Figure 6:change value to make error.

```
# KERNEL: time = 126728 | A = 63 | B = 56 | E = 0 | G = 1 | L = 0
# KERNEL: time = 126759 | A = 63 | B = 57 | E = 0 | G = 1 | L = 0
# KERNEL: time = 126790 | A = 63 | B = 58 | E = 0 | G = 1 | L = 0
# KERNEL: time = 126821 | A = 63 | B = 59 | E = 0 | G = 1 | L = 0
# KERNEL: time = 126852 | A = 63 | B = 60 | E = 0 | G = 1 | L = 0
# KERNEL: time = 126883 | A = 63 | B = 61 | E = 0 | G = 1 | L = 0
# KERNEL: time = 126914 | A = 63 | B = 62 | E = 0 | G = 1 | L = 0
# KERNEL: time = 126945 | A = 63 | B = 63 | E = 0 | G = 1 | L = 0
# KERNEL: time = 126960 | A = 63 | B = 63 | E = 1 | G = 1 | L = 0
# KERNEL: Fail Number of errors 127
```

Figure 7:the error simulation when making error.

As you can see the result doesn't true because we change just one gate.

Future work

My code has many advantages.First, The use of registers and D flip-flops ensures that inputs are properly latched and stored, providing stability to the comparison process. This helps in synchronizing with the clock signal and resetting the circuit when needed.After that,the test_circuit module serves as a testbench for verifying the functionality of the comparator. It includes initial blocks to generate clock signals and monitor output, which aids in testing and validation.Finally,The inclusion of both behavioral and structural approaches in the behavioral and Unsigned modules allows for comparison of different coding styles and their performance.

On the other hand,my code has some disadvanteges like Hard-coded Delays that the use of hard-coded delays (e.g., #(10), #(4), #(9)) in the modules might not be ideal for all applications, as these values may need adjustment based on the specific timing requirements of different systems.And the testbench checks for equality, greater than, and less than conditions but does not test edge cases or invalid inputs. More comprehensive test cases could enhance validation and robustness.So for that we can improve put code if this techniques will be good.

Conclusion

The provided Verilog code offers a comprehensive implementation of a 6-bit comparator, demonstrating both signed and unsigned comparisons. The modular design enhances readability and maintenance, while the testbench aids in validating the functionality. However, the complexity and use of hard-coded delays could be improved for broader applicability. Overall, the code effectively showcases the principles of digital comparison and serves as a valuable resource for understanding and implementing comparators in digital systems,and this code in future if we improve it more and more,then we can put it in devices,which it build about embedded systems.In other words,we can use it in simple ALU in their Cpu’s.